



GSoC 2026 Proposal – OWASP BLT

BLT-MCP & BLT-CLI — AI Agent Interface + Developer CLI for the BLT Ecosystem

Rebhi Mohamed Aziz

National School of Computer Science (ENSI), Manouba, Tunisia

About Me

Final-year CS student at ENSI (Tunisia). Engineering Degree in Computer Science (Expected Graduation: October 2026). I work on MCP servers, RAG pipelines, and backend APIs.

Professional Experience:

- **Sofrecom Tunisie (Feb–Apr 2026, current):** Building a scalable RAG system — Celery, LangGraph, RAGAS, pgvector/PostgreSQL, bi-encoder reranking. Wraps up before GSoC starts.
- **Sofrecom Tunisie (Jul–Aug 2025):** Built a PostgreSQL MCP server — natural language to CRUD queries, tested with Ollama DeepSeek, integrated with GitHub Copilot. This is the main reason I'm confident about BLT-MCP.
- **Actia Engineering Services (Jul–Sep 2024):** Backend API development with Angular and Spring Boot.

Certifications: Agentic AI Course | AI Agents in LangGraph (LangChain + Tavily)

Contact

- **Mailing List:** Subscribed to the OWASP BLT mailing list
 - **OWASP Slack:** Aziz Rebhi — active in [#project-blt](#)
-

Coding Skills

Development Platform

Component	Details
Operating System	Ubuntu Linux (22.04 LTS)
CPU	Intel Core i7
RAM	16 GB
Storage	512 GB SSD
Editor	VS Code with Python, ESLint, and Docker extensions
Version Control	Git & GitHub
Containers	Docker & Docker Compose

Programming Experience

Language	Level	Experience
Python	Intermediate	FastAPI, MCP server, RAG pipelines
JavaScript/TypeScript	Beginner	React basics, GitHub API integrations
Java	Beginner	Spring Boot backend at Actia internship
SQL	Intermediate	PostgreSQL, MySQL — schema design, query tuning, migrations
C / C++	Basic	University coursework

Relevant Knowledge Areas

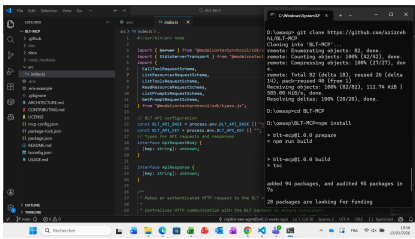
- **Backend:** Python, FastAPI, Django, REST APIs, JSON-RPC 2.0, MCP Python SDK
 - **AI/ML:** LangGraph, LangChain, RAG pipelines, ChromaDB, pgvector, Ollama, HuggingFace
 - **DevOps:** Docker, GitHub Actions CI/CD, OAuth 2.0, pytest
 - **Frontend:** React, Angular, Tailwind CSS
-

Environment Setup

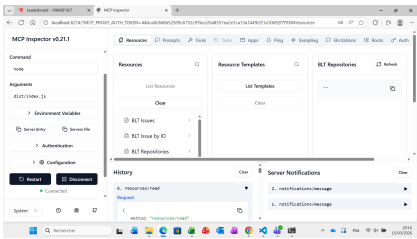
Cloned and set up all four BLT repositories locally: **BLT** (Django platform, 479 tests passing), **BLT-MCP** (built and tested with MCP Inspector), **BLT-Pages** (React frontend), and **BLT-API** (auth and API layer — contributed a bug fix PR here).

Screenshots — BLT-MCP running locally via MCP Inspector:

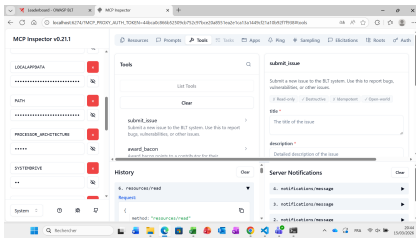
BLT-MCP Cloned & Built



MCP Inspector — Connected



MCP Inspector — Capabilities



Key observations: BLT-API (api.owaspblt.org) serves bugs, users, domains, and contributions. The MCP server wraps these via JSON-RPC 2.0. Auth uses API keys + OAuth.

Community Involvement

Past Contributions to OWASP BLT

7 merged PRs across 4 BLT repositories:

Repository	PR	Type	Description
BLT	#5916	Documentation	Fix Poetry setup order in README — fixed onboarding failures for new contributors
BLT	#5917	Refactoring	Optimize imports and add type hints to API views (website/api/views.py)
BLT-MCP	#21	Refactoring	Consolidate resource request logic in MCP handler — reduced duplication in <code>ReadResourceRequestSchema</code> , directly improved the codebase this GSoC project extends
BLT-API	#59	Bug Fix	Fix HTTP status code (404 → 405) in auth handlers — critical for MCP server to interpret API responses correctly
BLT-API	#67	Bug Fix	Add <code>headers</code> parameter to <code>error_response()</code> — fixed <code>TypeError</code> on 405 responses with <code>Allow</code> header
BLT-API	#68	Feature	Add <code>/routes</code> endpoint for programmatic API discoverability — returns all registered routes as structured JSON, directly useful for MCP tool/resource discovery
BLT-Pages	#28	Feature	Display GitHub issue reactions on bug cards

Most relevant: **PR #21** (MCP handler refactor), **PR #68** (API route discovery for MCP), and **PR #59** (the API layer the MCP server consumes).

Other Open-Source & Personal Projects

- **LangGraph Essay Writer** ([GitHub](#)) — Agentic essay system using LangGraph multi-step reasoning, Tavily research, Supabase state, Streamlit UI.
 - **ARE-Nexus** ([are-nexus-ql7t.vercel.app](#)) — University robotics platform with React frontend and a RAG chatbot (FastAPI + ChromaDB + OpenAI).
-

Why This Idea & What I Expect

1. **I've already built an MCP server.** PostgreSQL MCP at Sofrecom — natural language to SQL, tested with Ollama DeepSeek, integrated with GitHub Copilot. I know what the work involves.
2. **I understand how AI agents consume tools.** MCP + RAG pipelines + LangGraph — I know how agents discover and use tool APIs.
3. **I'm already in the codebase.** PR #21 on BLT-MCP — I've read through the handler architecture and protocol flow. I'm proposing migration to Python (official MCP SDK, no framework) for token efficiency and security-first design.

What this enables for BLT: AI agents can triage vulnerabilities, manage issues, and track contributions autonomously through MCP, while developers get fast terminal access via BLT-CLI — making BLT accessible from every interface: web, AI, and command line.

Project Proposal

Project Goals

The goal is to build a unified MCP interface that gives AI agents and developers complete access to BLT — reads and writes — through BLT-API. BLT-API already supports reads for all entities and writes for bug creation and auth; remaining write endpoints will be added as community projects. As those endpoints land, this project wraps them as MCP tools and CLI commands:

1. BLT-MCP — A production-ready MCP server exposing BLT to AI agents through the Model Context Protocol:

- **Resources** — Read-only access to BLT data (issues, repos, contributors, leaderboards, rewards)
- **Tools** — Actions agents can take (submit issues, update statuses, award points, manage comments)
- **Prompts** — Reusable workflow templates (vulnerability triage, contribution review, remediation planning)

2. BLT-CLI — A developer-facing command-line tool for interacting with BLT from the terminal:

- Built in Python with `click` (commands) + `rich` (colored tables, progress bars, ASCII animations)
- Shares the same API client library as the MCP server — one codebase, two interfaces
- Commands map to the same operations: `blt issues list`, `blt issue submit`, `blt rewards balance`, `blt leaderboard`
- Fun ASCII startup animation (as requested by @DonnieBLT)

Both interfaces share a **common Python API client layer** (auth, HTTP calls, response models), so code is written once and used by both MCP and CLI.

Maintainer Alignment

Both the Python migration and CLI inclusion were discussed with @DonnieBLT (project lead) on [#project-blt](#). He initiated the migration direction, approved the no-framework approach for minimal token usage, and requested the CLI be part of the proposal. My PoC PR ([#22](#)) was reviewed and scoped as GSoC work.

Priority & Fallback Scope

BLT-MCP is the primary deliverable. If the MCP migration takes longer than expected, MCP wins and CLI scope is reduced.

Minimum viable CLI (fallback): If time runs short, the shippable CLI v1 is 4 core read-only commands:

- `blt login` — OAuth authentication
- `blt issues list` — list issues with filters
- `blt issue show` — display full issue details
- `blt search` — full-text search across issues

These are the highest-value commands and the simplest to implement since they reuse the shared client without needing write-path complexity. The remaining commands (`blt issue submit`, `blt leaderboard`, `blt rewards balance`, `blt contributors show`, `blt repos list`) are delivered if schedule permits.

Shared client timeline: The shared API client is built in Weeks 1–2 and exercised throughout Weeks 3–8 by the MCP server. By the time CLI work begins in Week 9, it's already battle-tested — not a fresh dependency.

Proposed Migration to Python

After studying the existing TypeScript BLT-MCP codebase and the broader MCP ecosystem, I'm proposing to **migrate BLT-MCP from TypeScript to Python** using the **official MCP Python SDK — no framework approach**. Here's why:

- **Control:** Using the SDK directly instead of something like FastMCP means we see exactly how tools, resources, and prompts get registered and invoked. No magic, no hidden behavior. We own the full lifecycle.
- **Token efficiency:** Controlling serialization means stripping fields the LLM doesn't need, implementing tiered response formats (summary vs. full), and keeping responses compact. Fewer tokens = cheaper and faster agent interactions.
- **Security-first design:** Since this is an OWASP project, the server needs input validation, scoped auth, and rate limiting baked in from day one — not bolted on at the end. A no-framework approach means we control exactly what goes through the auth pipeline.
- **Same language as BLT:** The main platform is Django/Python. Keeping the MCP server in Python too means future contributors only need to know one stack.
- **AI ecosystem fit:** Python is where LangChain, LangGraph, and most agent frameworks live. A Python MCP server plugs in more naturally.

- **Async-native:** The MCP server uses `asyncio` for HTTP and SSE transports (concurrent connections) and `anyio` for stdio compatibility. The shared API client uses `httpx.AsyncClient` so all BLT-API calls are non-blocking.

Proven patterns from PostgreSQL MCP:

The PostgreSQL MCP server I built at Sofrecom follows the same no-framework approach using the official MCP Python SDK. Key patterns I'll reuse:

- Error boundary decorator that catches exceptions, logs them, and returns structured JSON (prevents crashes from malformed API responses)
- Automatic LIMIT injection on unbounded SELECT-style operations (prevents token overflow)
- Dual-format responses (raw + markdown) optimized for LLM consumption

I submitted a proof-of-concept PR (#22) that converts the existing TypeScript server to a minimal Python version — implementing the issues resource, a `submit_issue` tool, and a `triage_vulnerability` prompt using the official MCP Python SDK with no framework. That'll be the starting point for the GSoC work.

Migration Transition Strategy

The Python server **fully replaces** the TypeScript version — no parallel deployment. Phase 1 focuses on feature parity: every existing TS handler gets a Python equivalent, verified by the same protocol compliance tests. A **parity checklist** (spreadsheet mapping each TS handler → Python equivalent → test status) is maintained throughout and shared with the mentor weekly. The TypeScript code stays in the repo as reference until the Python version passes all tests and every checklist row is green, then it's removed. The last TS commit is tagged (`ts-final`) before removal so it can be restored within one working day if the Python server has a critical regression post-cutover. Clean cutover once parity is confirmed. This avoids maintaining two codebases while ensuring no existing functionality is lost.

BLT-CLI — Developer Command-Line Tool

BLT-CLI gives developers terminal access to BLT using the same shared API client as BLT-MCP:

Core commands:

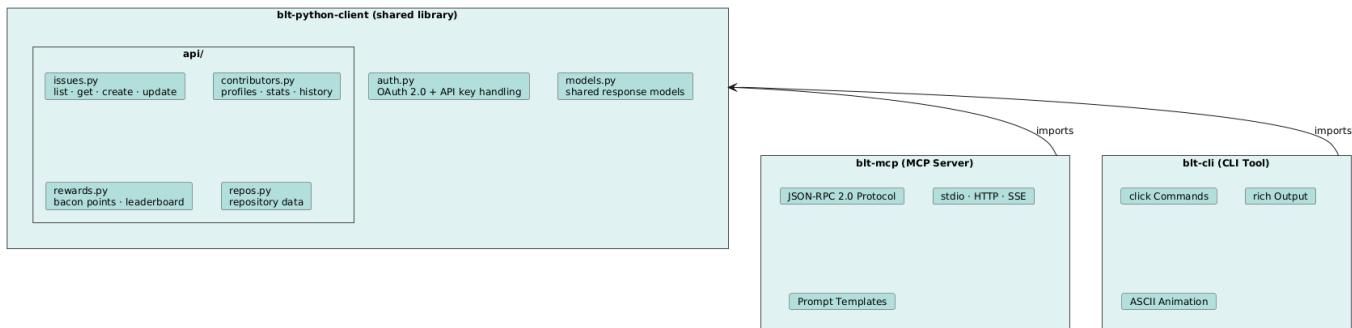
Command	Description
<code>blt issues list --status open</code>	List issues with filters (status, severity, repo)
<code>blt issue show 42</code>	Show full issue details
<code>blt issue submit --title "XSS in login"</code>	Submit a new bug report
<code>blt search "SQL injection"</code>	Full-text search across issues
<code>blt leaderboard</code>	Show top contributors
<code>blt rewards balance</code>	Check your bacon points
<code>blt contributors show azizrebhi</code>	View contributor profile + stats
<code>blt repos list</code>	List tracked repositories

Tech stack: Python, `click` (CLI framework), `rich` (colored tables, panels, spinners, ASCII art)

Developer experience details:

- **Auth:** `blt login` triggers OAuth and stores the token in `~/.config/blt/credentials.json`. Subsequent commands read it automatically.
- **Config:** `~/.config/blt/config.toml` stores defaults (API base URL, output format, pagination size). `blt config set` updates values.
- **Error handling:** Proper exit codes (0 = success, 1 = user error, 2 = API error), human-readable messages, `--verbose` for full HTTP debugging, `--json` flag for machine-readable output (useful for scripting and CI pipelines).
- **Offline behavior:** Graceful degradation if the BLT API is unreachable; read commands can fall back to cached data.

Shared architecture:



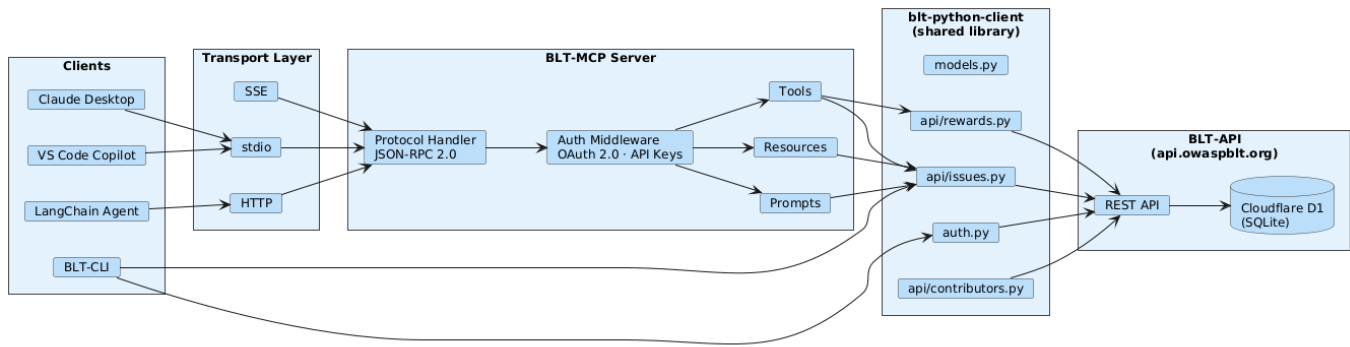
Success Criteria

Criterion	Target	Verification
Python migration	Feature parity with TS version	Side-by-side protocol compliance tests pass
MCP Resources	10+ resources with standardized URIs	Integration tests validate response schemas against BLT API
MCP Tools	10+ tools (issues, rewards, contributions)	Each tool has an end-to-end test in CI that verifies the response matches the MCP tool result schema and that side effects (e.g., issue created, status changed) actually persist in BLT
MCP Prompts	5+ workflow templates	Tested with Claude and LangChain agents
Transports	stdio, HTTP, SSE all functional	MCP Inspector validates each; stdio also tested with VS Code Copilot / Claude Code
Authentication	OAuth 2.0 + API key, scoped per tool	Auth bypass tests — unauthorized calls must return 403
Token efficiency	Paginated issues list <2K tokens in summary mode	Benchmark test in CI that fails if budget exceeded

Criterion	Target	Verification
Test coverage	>80% code coverage	Enforced in GitHub Actions — PR blocked if coverage drops
Documentation	Getting started, API ref, security model, examples	Reviewed by mentor before final submission
BLT-CLI	4 core commands minimum (login, issues list, issue show, search); 8+ if schedule permits	Each command tested in CI, output validated
Shared client	Common Python API client used by both MCP and CLI	Import tests verify both projects use same client

Proposed System Architecture

Here's how the pieces fit together:



Transports: stdio (local CLI/VS Code), HTTP (web/remote agents), SSE (real-time subscriptions)

Auth: OAuth 2.0 for user operations, API keys for service calls, scoped permissions per tool

MCP Request Flow

How a tool call travels through the system — from AI agent to BLT-API and back, shown as four focused sequence diagrams:

Figure 1 — Authentication & Authorization

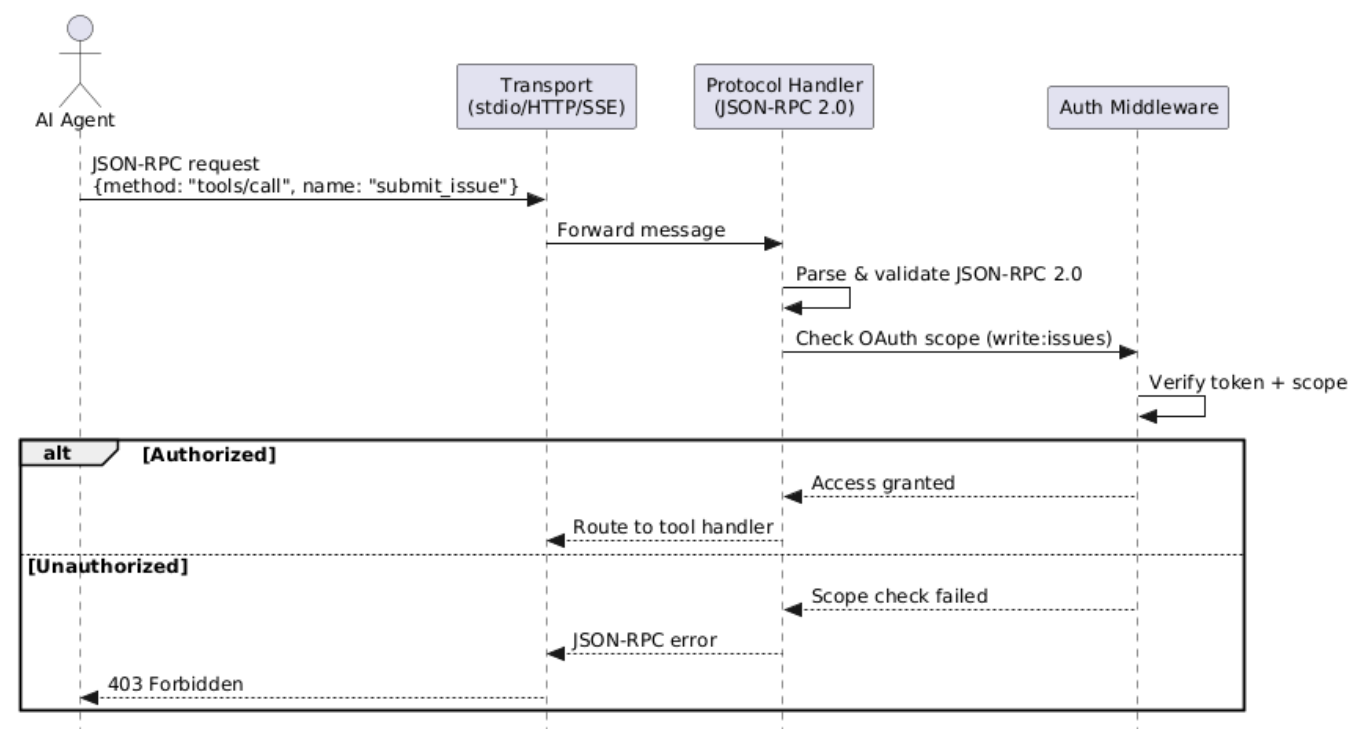


Figure 2 — Tool Execution (submit_issue)

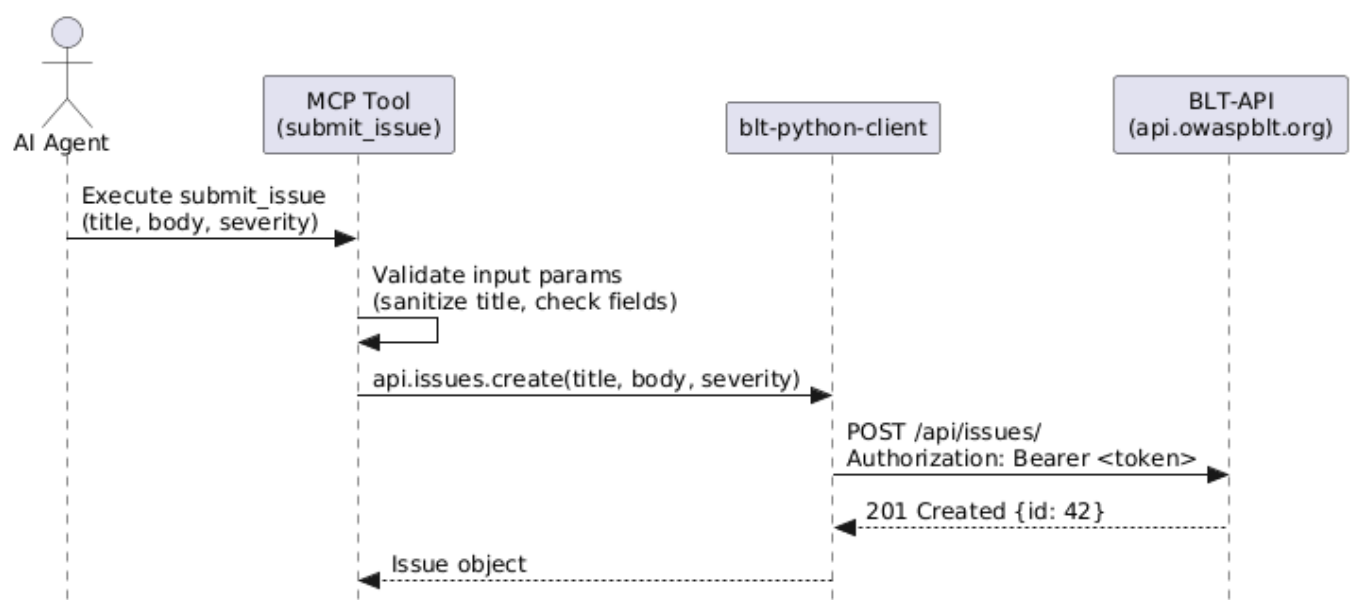


Figure 3 — Response Formatting & Token Budgeting

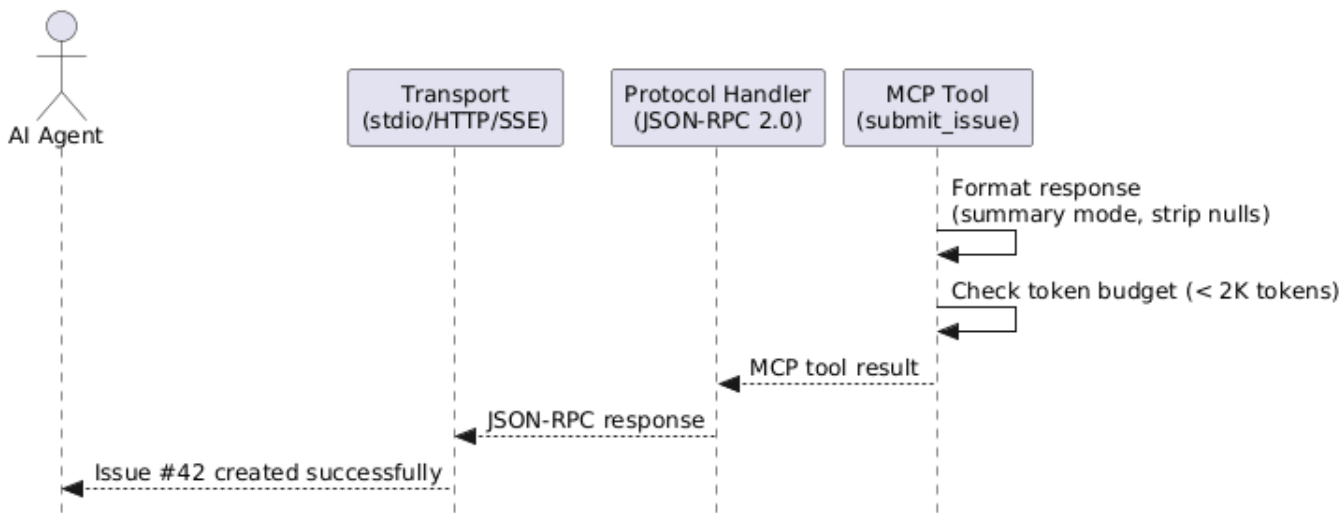
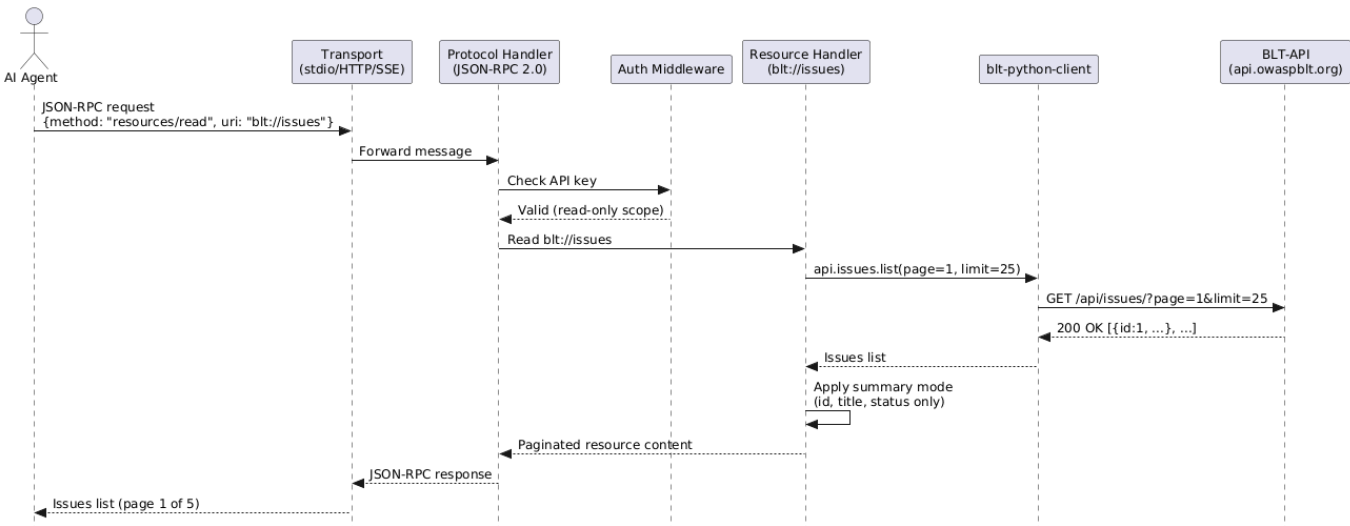


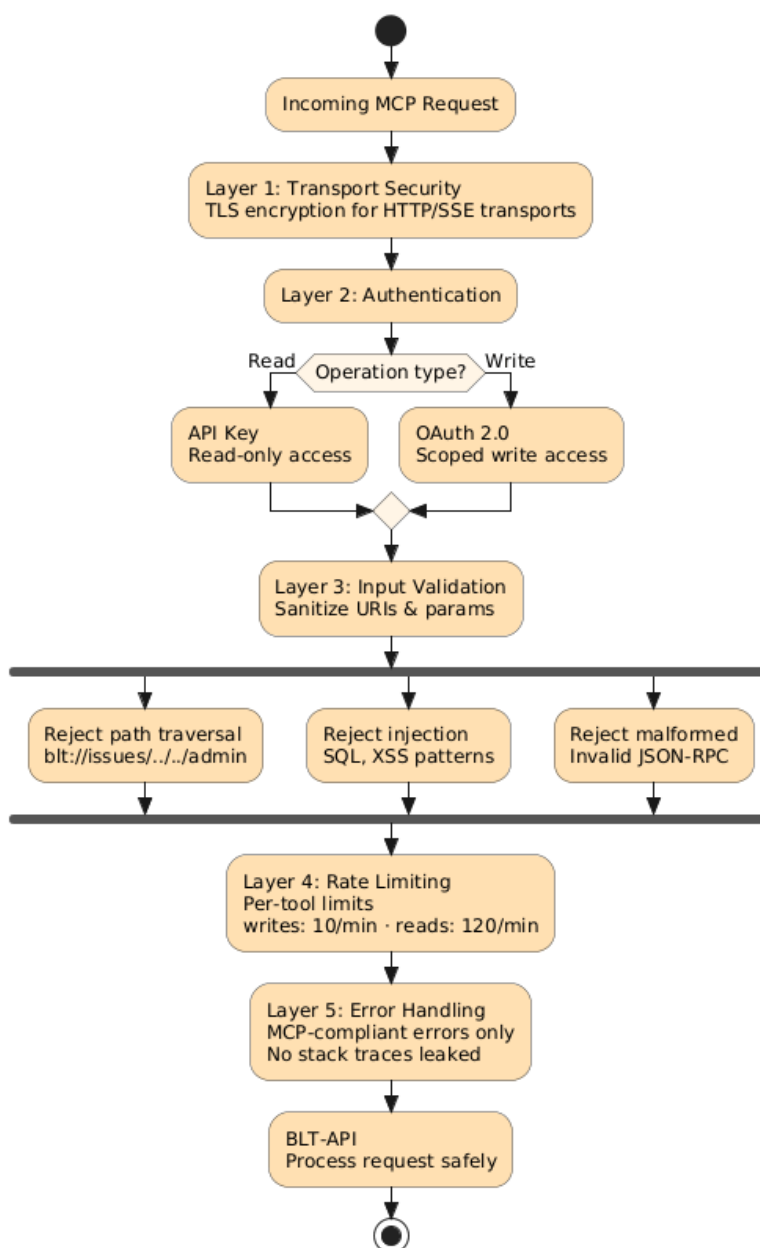
Figure 4 — Read Resource Flow (blt://issues)



Security Model

Since this is an OWASP project, security is built into every phase — not a final-week add-on:

Security Model — Request Processing Layers



Layer	Implementation
Scoped auth	Read-only via API key; writes require OAuth 2.0 with specific scopes
Rate limiting	Configurable per-tool limits — writes: 10 req/min (e.g., <code>submit_issue</code> , <code>award_bacon</code>); reads: 120 req/min per key. Enforced via sliding-window counter in-memory, configurable per deployment
Input validation	Sanitize all resource URIs and tool parameters; reject path traversal in resource URIs (e.g., <code>blt://issues/../../admin</code>), SQL injection in search params, and malformed JSON-RPC payloads before they reach BLT API

Layer	Implementation
Error handling	MCP-compliant error codes only — no stack traces, no internal details leaked. Errors log full tracebacks to disk for debugging but return only sanitized MCP error codes to clients
Final audit (Week 12)	Injection testing, auth bypass attempts, resource enumeration checks

Core Components

Resources (read-only data via standardized URIs):

Resource URI	Returns
<code>blt://issues</code> , <code>blt://issues/{id}</code>	Issues list (paginated) / single issue details
<code>blt://repos</code> , <code>blt://repos/{name}</code>	Repositories list / repo details + stats
<code>blt://contributors/{id}</code>	Contributor profile, reputation, history
<code>blt://leaderboard</code>	Current leaderboard rankings
<code>blt://rewards</code>	Available rewards and point balances

Tools (actions agents can take):

Tool Name	Description
<code>submit_issue</code>	Create a new bug report / issue
<code>update_issue_status</code>	Change issue status (open/closed/triage)
<code>add_comment</code>	Add a comment to an existing issue
<code>assign_issue</code>	Assign an issue to a contributor
<code>award_bacon</code>	Award bacon points to a contributor
<code>search_issues</code>	Full-text search across issues
<code>get_contributor_stats</code>	Fetch detailed contributor statistics
<code>flag_duplicate</code>	Flag an issue as a duplicate
<code>submit_pr_review</code>	Submit a review for a pull request

Tool prioritization: If schedule pressure arises, tools are prioritized: Tier 1 (`submit_issue`, `update_issue_status`, `search_issues`, `award_bacon`), Tier 2 (`add_comment`, `assign_issue`, `flag_duplicate`), Tier 3 stretch (`get_contributor_stats`, `submit_pr_review`). Additional tools (e.g., `follow_user`, `unfollow_user`) are added as their BLT-API endpoints land.

Prompts (workflow templates):

Prompt Name	Description
triage_vulnerability	Analyze issue severity, suggest priority and labels
plan_remediation	Generate a remediation plan for a security issue
review_contribution	Evaluate contribution quality and suggest rewards
weekly_summary	Generate activity summary for a repository
onboard_contributor	Provide onboarding guidance for new contributors

Potential Challenges & Mitigations

Based on building the PostgreSQL MCP server at Sofrecom and working with BLT's codebase, here are the issues I expect to run into — and how I plan to handle them:

1. Context Window Overflow from Large Responses

In my PostgreSQL MCP server, a `get_schema` tool returning the full database schema consumed most of the context window on large databases, degrading LLM reasoning. The same risk applies to BLT-MCP — unbounded `blt://issues` or `blt://contributors` calls could return 50K+ tokens.

Mitigations:

- **Paginated responses** — 10–25 items per page by default; agents request more as needed
- **Tiered response formats** — `summary` mode (id, title, status) vs `full` mode; default to summary
- **Selective field inclusion** — agents specify which fields they need (e.g., `fields=["id", "title", "severity"]`)
- **Response size budget** — configurable max token estimate; truncate with `has_more` flag if exceeded
- **Lean serialization** — strip null fields, short key names, no nested duplication

Proven in PostgreSQL MCP: Redesigned the schema tool to return table names first, then individual schemas on demand. Also applied auto-limit enforcement (`LIMIT 100` on unbounded queries), `limit/offset` pagination with `has_more` flags, and dual-format responses (JSON + markdown tables).

2. Other Challenges

Challenge	Mitigation
Token efficiency vs. completeness	Compact default response format, full details on explicit request, benchmark token counts per response type
Auth across three transports	Unified auth middleware at protocol handler level — abstracts stdio/HTTP/SSE differences
BLT API availability	Response caching with TTL, retry with exponential backoff + jitter (max 3 retries, base 500 ms), graceful degradation with staleness indicators
BLT-API write endpoint availability	BLT-API has reads for all entities but only a few write endpoints today (create bug, auth). Remaining writes will be added as community projects. Cross-project coordination on #project-blt ensures new endpoints are wrapped as MCP tools promptly; tools for not-yet-available writes are stubbed with clear "coming soon" errors
MCP spec evolution	Targets MCP protocol version 2025-03-26 . Thin protocol abstraction layer so spec bumps only require changes in one place; any mid-summer update is absorbed during Friday buffers

Schedule

I'll follow a PR-driven approach — each feature as a small, reviewable PR with tests and docs.

Day-by-Day Timeline

Community Bonding (May 26 – Jun 8)

Day	Task
Mon	Clone existing TS BLT-MCP repo, read through every handler file
Tue	Document all existing MCP handlers and their BLT API mappings
Wed	Read MCP spec — protocol lifecycle, capability negotiation, JSON-RPC format
Thu	Read MCP spec — resources, tools, prompts, transport requirements
Fri	Map existing TS handlers to planned Python equivalents in a spreadsheet
Mon	Design shared blt-python-client API surface (auth, issues, contributors, rewards)
Tue	Draft architecture document: shared client + MCP server + CLI structure
Wed	Review architecture with mentor, incorporate feedback
Thu	Set up dev environment: Python 3.11+, MCP Python SDK, pytest, ruff, mypy
Fri	Finalize design document + migration strategy, submit for mentor review

Week 1 — Project Setup + Shared API Client Foundation

Day	Task
Mon	Create <code>pyproject.toml</code> , project structure, GitHub Actions CI skeleton
Tue	Implement base HTTP client with retry logic and error handling
Wed	Build <code>issues</code> API module — list, get, create, update via BLT-API
Thu	Scaffold BLT-CLI project (<code>click</code> app, <code>blt --help</code> , config file loading), write unit tests for HTTP client + issues module
Fri	Buffer — absorb setup blockers, mentor sync, verify CI passes

Week 2 — Auth Module + MCP Protocol Handler + stdio Transport

Day	Task
Mon	Build auth module — OAuth 2.0 token flow (authorization code grant, token refresh, secure storage)
Tue	Add API key header injection, scoped permissions; write auth unit tests
Wed	Initialize MCP server using official Python SDK (no framework), register capabilities
Thu	Implement JSON-RPC 2.0 message parsing, add stdio transport
Fri	Migrate first TS handler to Python: issues list resource; run MCP Inspector against stdio

Week 3 — HTTP/SSE Transports + Token Efficiency

Day	Task
Mon	Implement HTTP transport with proper request/response handling
Tue	Implement SSE transport for real-time subscriptions
Wed	Build tiered response format — summary mode (id, title, status) vs full mode
Thu	Add selective field inclusion and response size budget logic
Fri	Buffer — compliance tests against TS version, fix issues, verify feature parity on all transports

Week 4 — MCP Resources: Issues + Repos

Day	Task
Mon	Implement <code>blt://issues</code> resource with pagination (default 25 items)
Tue	Implement <code>blt://issues/{id}</code> resource with full issue details
Wed	Implement <code>blt://repos</code> and <code>blt://repos/{name}</code> resources
Thu	Add response caching with configurable TTL for all resources
Fri	Buffer — integration tests, schema validation against BLT API, address mentor feedback

Week 5 — MCP Resources: Contributors, Leaderboard, Rewards

Day	Task
Mon	Implement <code>blt://contributors/{id}</code> resource with profile + history
Tue	Implement <code>blt://leaderboard</code> resource with ranking data
Wed	Implement <code>blt://rewards</code> resource with point balances
Thu	Add input sanitization on all resource URI parameters
Fri	Buffer — integration tests for new resources, edge cases, catch-up on any Week 4 overflow

Week 6 — Token Optimization + Midterm Prep

Day	Task
Mon	Benchmark token count for every resource in summary vs full mode
Tue	Add CI test that fails if any summary response exceeds 2K tokens
Wed	Strip null fields, optimize JSON serialization across all responses
Thu	Prepare midterm demo — run full MCP server with all resources via Inspector
Fri	Midterm evaluation — all resources complete, demo-ready, tests green

Week 7 — MCP Tools: Issue Management

Day	Task
Mon	Implement MCP <code>submit_issue</code> tool with OAuth scope check (BLT-API create-bug endpoint exists)
Tue	Implement MCP <code>update_issue_status</code> tool (consumes BLT-API endpoint once BLT-Next ships it; stubbed until then)
Wed	Implement MCP <code>add_comment</code> tool with input validation
Thu	Implement MCP <code>assign_issue</code> tool with contributor existence check
Fri	Buffer — end-to-end tests, unauthorized access attempts, mentor feedback

Week 8 — MCP Tools: Rewards, Search, Stats

Day	Task
Mon	Implement MCP <code>award_bacon</code> tool with OAuth scope check
Tue	Implement <code>search_issues</code> MCP tool with query sanitization (BLT-API search endpoint exists)
Wed	Implement <code>get_contributor_stats</code> and <code>flag_duplicate</code> tools
Thu	Implement <code>submit_pr_review</code> tool, add rate limiting on all write tools
Fri	Buffer — full tool test suite, auth/rate-limit/malformed-input cases, catch-up

Week 9 — BLT-CLI: Core Commands (*scaffolding done in Week 1, shared client battle-tested in Weeks 2–8*)

Day	Task
Mon	Wire <code>blt login</code> OAuth flow using existing auth module, build ASCII startup animation
Tue	Implement <code>blt issues list</code> with colored table output and filters
Wed	Implement <code>blt issue show</code> with formatted panel display
Thu	Implement <code>blt search</code> with highlighted results
Fri	Buffer — error handling (exit codes, <code>--verbose</code>), offline fallback, CLI tests

Week 10 — BLT-CLI: Full Commands + MCP Prompts Start

Day	Task
Mon	Implement <code>blt issue submit</code> with interactive prompts for required fields
Tue	Implement <code>blt leaderboard</code> and <code>blt rewards balance</code> with rich tables
Wed	Implement <code>blt contributors show</code> and <code>blt repos list</code>
Thu	Build MCP prompt: <code>triage_vulnerability</code> — severity analysis template
Fri	Build MCP prompt: <code>plan_remediation</code> — security fix planning template

Week 11 — MCP Prompts + Testing with AI Agents

Day	Task
Mon	Build prompts: <code>review_contribution</code> and <code>weekly_summary</code>
Tue	Build prompt: <code>onboard_contributor</code> — new contributor guidance template
Wed	Test all prompts with Claude Desktop — verify output quality
Thu	Test prompts with LangChain agent — verify tool discovery and chaining
Fri	CLI polish: error messages, edge cases, help text, config file support

Week 12 — Security Audit + Documentation + Final Polish

Day	Task
Mon	Security audit: injection testing on all tool inputs and resource URIs
Tue	Security audit: auth bypass attempts, calling write tools with read-only keys
Wed	Security audit: resource enumeration checks, error message review
Thu	Write documentation: getting started guide, API reference, CLI usage, security model
Fri	Buffer — final test pass (>80% coverage), tag release, submit final evaluation

Weekly cadence: Every week I will submit at least one PR and provide a written progress report via Slack and GitHub.

Testing & CI Pipeline

GitHub Actions runs on every PR — nothing merges without passing:

- **Unit tests** (pytest) — each resource, tool, and prompt handler tested in isolation. Write-path tools like `award_bacon` are tested against a stubbed BLT API (responses recorded from the real API, replayed via `pytest-httpx`) so tests run without a live rewards backend. Any upstream API changes are caught by weekly integration test runs against a local BLT instance.
- **Integration tests** — against BLT-API, verifying the full MCP-to-API request path
- **MCP protocol compliance** — MCP Inspector validates JSON-RPC format, capability negotiation, and all three transport behaviors
- **Token budget tests** — enforce response size limits (fail if summary response exceeds 2K tokens, counted via `tiktoken` with `cl100k_base` encoding)
- **Coverage gate** — 80% minimum, PR blocked if coverage drops; new code always comes with tests
- **Linting + type checking** — `ruff` + `mypy` on every commit, catches bugs early and keeps the codebase consistent for future contributors

Deliverables

By the end of the project, the following will be delivered:

#	Deliverable	Status Criteria
1	Python BLT-MCP server (migrated from TypeScript) using official MCP Python SDK, no framework approach	Passes all protocol compliance tests, feature parity with TS version
2	stdio, HTTP, and SSE transport support	All three transports tested and functional
3	MCP Resources for all BLT entities	10+ resources with pagination and caching
4	MCP Tools for BLT actions	10+ tools with input validation; consumes existing and incoming BLT-API endpoints
5	MCP Prompts for workflow automation	5+ prompt templates tested with AI agents
6	OAuth 2.0 and API key authentication	Integrated from Phase 1, scoped per tool
7	Security: input validation, rate limiting, audit	No injection vectors, no data leaks, no auth bypass
8	Test suite with >80% code coverage	CI/CD integration with GitHub Actions
9	Developer documentation + security model docs	Getting started, API reference, security docs, examples
10	Integration examples	Working examples for Claude, LangChain, VS Code

#	Deliverable	Status Criteria
11	BLT-CLI with <code>click</code> + <code>rich</code>	4 core commands minimum; 8+ if schedule permits. ASCII animation, colored output, tested in CI
12	Shared Python API client library	Used by both MCP server and CLI, published as internal package

Why I'm Suited for This Project

Requirement	Experience
MCP protocol	Built MCP server at Sofrecom + contributed to BLT-MCP (PR #21)
Python backend	Python — MCP server, RAG system, FastAPI APIs
AI agent integration	Integrated MCP with Ollama DeepSeek & GitHub Copilot; built RAG pipelines
BLT ecosystem	Contributed across all 4 BLT repos
Auth / Security	Implemented OAuth 2.0 in production systems
CLI development	Built CLI tools with Python; familiar with <code>click</code> and <code>rich</code>
Token efficiency & error handling	Built <code>safe_tool</code> decorator pattern with structured error responses, enforced result limits, and pagination in PostgreSQL MCP — directly applicable to BLT-MCP's token budget challenges

AI Tools Disclosure

I use AI tools to accelerate development, not to replace understanding:

- **GitHub Copilot** — code completion and boilerplate generation in VS Code. I review every suggestion, reject anything that doesn't fit the architecture, and never commit code I don't understand.
- **Claude** — for brainstorming design decisions (e.g., tiered response format trade-offs), drafting documentation, and reviewing my own code for security issues.
- **How I avoid AI slop:** I write the architecture and core logic myself, then use AI for repetitive parts (serializers, test boilerplate, docstrings). Every AI-generated line gets reviewed against the MCP spec and BLT API docs. If it hallucinates an API endpoint or protocol detail, I catch it because I've read the spec.

Time Commitment

Period	Hours/Week	What I'll be doing
Before GSoC	8–10	More PRs, studying the existing TypeScript repo, reading the MCP spec closely (my current Sofrecom internship wraps up in April)
During GSoC	30–35	Full-time development, weekly PRs, progress reports
After GSoC	5–8	Maintenance, new features, helping other contributors

No conflicting commitments during the coding period (June–August 2026).

Communication: `#project-blt` on Slack (daily), GitHub PRs (weekly), written progress reports.



OWASP BLT × Google Summer of Code 2026